C++ ARRAYS

Professional Study Notes and Practical Workbook

For Beginner to Intermediate C++ Students

University-oriented notes • Complete syntax • 10 practical examples per major topic • 25 practical scenarios with model solutions

Purpose

This guide develops a rigorous understanding of arrays in C++. It starts with the language fundamentals and progressively introduces one-dimensional arrays, multidimensional arrays, algorithms, functions, pointers, character arrays, std::array, and std::vector. Explanations are intentionally beginner-friendly, but the terminology, syntax, examples, and programming practices are presented at an academic and professional standard.

Learning outcomes

• Define an array and explain its relationship to indexed storage.

• Declare, initialize, access, update and traverse arrays correctly.

• Write robust programs that process collections of numerical and textual data.

• Use arrays with loops, conditions, functions, pointers and structures.

• Implement common operations such as searching, sorting, aggregation and frequency counting.

• Work with two-dimensional arrays and basic matrix operations.

• Recognize limitations of built-in arrays and select std::array or std::vector appropriately.

• Identify and prevent common indexing, initialization, memory and boundary errors.

Chapter 1 — Array Fundamentals

1.1 Definition and purpose

An array is a fixed-size sequence of elements of the same type stored in contiguous memory. Each element is identified by an integer index. Arrays are useful when a program needs to process many related values using one logical name.

The general declaration syntax is:

```
data_type array_name[array_size];
```

For example, int marks[5]; declares five integer elements. Their indexes are 0 through 4. The expression marks[0] refers to the first element and marks[4] to the last.

> Critical rule: for an array containing N elements, the valid index range is 0 through N-1. Accessing N or any larger index is outside the array's bounds and produces undefined behavior for a built-in C++ array.

Example 1: Five examination marks

A fixed collection of five integer marks.

```
int marks[5];
```

Example 2: Product quantities

Stores quantities for eight products.

```
int quantity[8];
```

Example 3: Monthly revenue

Stores twelve decimal revenue values.

```
double revenue[12];
```

Example 4: Daily temperature

Stores seven temperature readings.

```
double temperature[7];
```

Example 5: Employee IDs

Stores six integer identifiers.

```
int employeeId[6];
```

Example 6: Attendance status

Stores ten Boolean values.

```
bool present[10];
```

Example 7: Character buffer

Reserves twenty character positions.

```
char buffer[20];
```

Example 8: Test scores

Stores four floating-point scores.

```
float scores[4];
```

Example 9: Inventory levels

Stores fifteen inventory quantities.

```
int inventory[15];
```

Example 10: Classroom capacities

Stores capacities for five rooms.

```
int capacity[5];
```

1.2 Declaration versus initialization

Declaration reserves storage and specifies the type and number of elements. Initialization supplies initial values. Keeping these concepts distinct helps prevent uninitialized-data errors.

Example 1: Declaration only

Storage is declared; values have not been assigned explicitly.

```
int marks[5];
```

Example 2: Full initialization

Every element receives a value.

```
int marks[5] = {72, 68, 81, 90, 77};
```

Example 3: Size inferred

When an initializer is supplied, the compiler can determine the size.

```
int values[] = {10, 20, 30, 40};
```

Example 4: Partial initialization

Unspecified remaining elements are value-initialized to zero.

```
int values[5] = {10, 20};
```

Example 5: All zeros

An empty initializer value-initializes all elements.

```
int values[5] = {};
```

Example 6: Double array

Uses double precision floating-point values.

```
double price[4] = {12.50, 9.99, 30.00, 18.75};
```

Example 7: Character array

Stores individual characters.

```
char grades[5] = {'A', 'B', 'C', 'D', 'E'};
```

Example 8: Boolean array

Stores true/false states.

```
bool status[4] = {true, false, true, true};
```

Example 9: Constant-size array

const prevents the array size from being changed.

```
const int SIZE = 10;
int values[SIZE];
```

Example 10: Compile-time expression

The size can be a constant expression.

```
int values[2 * 5];
```

Chapter 2 — Indexing, Access and Modification

2.1 Array indexing

The subscript operator [] selects an individual element. The index is an expression that evaluates to an integer-like value. The first element has index zero.

Example 1: Read first element

Retrieves the first element.

```
cout << marks[0];
```

Example 2: Read last element

For five elements, index 4 is the last valid index.

```
cout << marks[4];
```

Example 3: Assign a variable

Copies an element into another variable.

```
int firstMark = marks[0];
```

Example 4: Modify an element

Assignment changes the selected element.

```
marks[2] = 95;
```

Example 5: Increment an element

Updates the selected value.

```
marks[1]++;
```

Example 6: Arithmetic expression

Array elements can participate in calculations.

```
int total = marks[0] + marks[1];
```

Example 7: Comparison

An element can be compared with a threshold.

```
if (marks[3] >= 50) cout << "Pass";
```

Example 8: Multiple access

Several elements can be used in one expression.

```
int total = marks[0] + marks[2] + marks[4];
```

Example 9: Use an index variable

A variable can determine which element is accessed.

```
int i = 3;
cout << marks[i];
```

Example 10: Boundary-safe access

Check the index before accessing an array.

```
if (i >= 0 && i < 5) cout << marks[i];
```

2.2 Traversal with loops

Traversal is the systematic processing of each element. A conventional for loop is especially suitable when the array size is known.

Example 1: Print all values

Processes indexes 0 through 4.

```
for (int i = 0; i < 5; ++i) cout << marks[i] << ' ';
```

Example 2: Print each on a new line

Adds a line break for each element.

```
for (int i = 0; i < 5; ++i) cout << marks[i] << endl;
```

Example 3: Sum values

Accumulates all elements into total.

```
int total = 0;
for (int i=0; i<5; ++i) total += marks[i];
```

Example 4: Count passes

Counts elements meeting a condition.

```
int count = 0;
for (int i=0; i<5; ++i) if (marks[i] >= 50) ++count;
```

Example 5: Double every element

Updates every element in place.

```
for (int i=0; i<5; ++i) marks[i] *= 2;
```

Example 6: Print indexes

Displays index and corresponding value.

```
for (int i=0; i<5; ++i) cout << i << ": " << marks[i] << endl;
```

Example 7: Reverse traversal

Visits elements from the final index to zero.

```
for (int i=4; i>=0; --i) cout << marks[i] << ' ';
```

Example 8: Conditional display

Displays only values above a threshold.

```
for (int i=0; i<5; ++i) if (marks[i] > 80) cout << marks[i] << ' ';
```

Example 9: Copy elements

Copies corresponding elements into another array.

```
for (int i=0; i<5; ++i) copy[i] = marks[i];
```

Example 10: Range-based for

Modern syntax processes each value directly.

```
for (int mark : marks) cout << mark << ' ';
```

Chapter 3 — Input, Output and Array Processing

3.1 Keyboard input

When the number of values is known, a loop can repeatedly read into array elements. The loop condition must match the valid index range.

Example 1: Read five marks

Stores five user-entered marks.

```
int marks[5];
for (int i=0; i<5; ++i) cin >> marks[i];
```

Example 2: Read ten integers

Collects ten integer values.

```
int a[10];
for (int i=0; i<10; ++i) cin >> a[i];
```

Example 3: Read six prices

Collects decimal prices.

```
double price[6];
for (int i=0; i<6; ++i) cin >> price[i];
```

Example 4: Read seven temperatures

Collects a week's readings.

```
double t[7];
for (int i=0; i<7; ++i) cin >> t[i];
```

Example 5: Prompt each input

Provides meaningful prompts.

```
for (int i=0; i<5; ++i) { cout << "Enter mark " << i+1 << ": "; cin >> marks[i]; }
```

Example 6: Read and validate basic range

Repeats until a mark is between 0 and 100.

```
for(int i=0;i<5;++i){ do{cin>>marks[i];}while(marks[i]<0||marks[i]>100); }
```

Example 7: Read into a constant-size array

Uses a named size constant.

```
const int N=5; int a[N];
for(int i=0;i<N;++i) cin>>a[i];
```

Example 8: Read and display

Combines input and output.

```
for(int i=0;i<3;++i) cin>>a[i];
for(int i=0;i<3;++i) cout<<a[i]<<' ';
```

Example 9: Read positive values

Rejects non-positive input.

```
for(int i=0;i<5;++i){ cin>>a[i]; if(a[i]<=0) --i; }
```

Example 10: Read multiple test scores

Stores one student's subject marks.

```
double score[6];
for(int i=0;i<6;++i) cin>>score[i];
```

3.2 Aggregate analysis

Common array-processing tasks include sum, mean, maximum, minimum, counts and derived statistics. These operations form the basis of many data-processing programs.

Example 1: Sum

Calculates the total.

```
int sum=0; for(int i=0;i<5;++i) sum+=marks[i];
```

Example 2: Average

Uses floating-point division to preserve decimals.

```
double average=(double)sum/5;
```

Example 3: Maximum

Initializes from the first element and compares the remainder.

```
int maximum=marks[0]; for(int i=1;i<5;++i) if(marks[i]>maximum) maximum=marks[i];
```

Example 4: Minimum

Finds the smallest element.

```
int minimum=marks[0]; for(int i=1;i<5;++i) if(marks[i]<minimum) minimum=marks[i];
```

Example 5: Pass count

Counts values at least 50.

```
int pass=0; for(int x:marks) if(x>=50) ++pass;
```

Example 6: Fail count

Counts values below 50.

```
int fail=0; for(int x:marks) if(x<50) ++fail;
```

Example 7: Even count

Counts even integers.

```
int even=0; for(int x:a) if(x%2==0) ++even;
```

Example 8: Positive count

Counts positive values.

```
int positive=0; for(int x:a) if(x>0) ++positive;
```

Example 9: Range

Computes maximum minus minimum.

```
int range=maximum-minimum;
```

Example 10: Weighted total

Multiplies corresponding values by weights.

```
double total=0; for(int i=0;i<5;++i) total+=marks[i]*weight[i];
```

Chapter 4 — Array Size and Memory Model

4.1 Determining element count

For a built-in array in a scope where it is still an array type, sizeof(array) gives the total storage in bytes. sizeof(array[0]) gives the size of one element. Their ratio gives the number of elements.

Example 1: Integer count

Calculates the number of int elements.

```
int a[5];
size_t n = sizeof(a) / sizeof(a[0]);
```

Example 2: Double count

Works for double arrays.

```
double a[12];
size_t n = sizeof(a) / sizeof(a[0]);
```

Example 3: Character count

Calculates allocated character positions.

```
char a[20];
size_t n = sizeof(a) / sizeof(a[0]);
```

Example 4: Loop using size

Avoids repeating a literal size.

```
for(size_t i=0;i<sizeof(a)/sizeof(a[0]);++i) cout<<a[i];
```

Example 5: Use constexpr

Defines a compile-time array size.

```
constexpr size_t N=8;
int a[N];
```

Example 6: Infer then calculate

Works with an initialized built-in array.

```
int a[]={2,4,6,8};
size_t n=sizeof(a)/sizeof(a[0]);
```

Example 7: Avoid sizeof on pointer

A pointer does not contain array length.

```
int* p=a;
// sizeof(p) is pointer size, not element count
```

Example 8: Function parameter warning

An array parameter is adjusted to a pointer.

```
void f(int a[]) { /* a is treated as pointer */ }
```

Example 9: Prefer explicit size in function

Pass the number of elements separately.

```
void f(const int a[], size_t n) { for(size_t i=0;i<n;++i) cout<<a[i]; }
```

Example 10: Modern alternative

std::size provides clearer intent for actual arrays.

```
#include <iterator>
size_t n=std::size(a);
```

Built-in arrays do not perform bounds checking. Modern C++ programs should strongly consider std::array for fixed-size collections and std::vector for dynamically sized collections.

Chapter 5 — Two-Dimensional and Multidimensional Arrays

5.1 Two-dimensional array syntax

A two-dimensional array represents data arranged in rows and columns. The syntax is data_type name[rows][columns]. Access uses array[row][column].

```
int marks[3][2] = {
    {72, 81},
    {64, 75},
    {90, 88}
};
```

The array above contains 3 × 2 = 6 elements. Valid row indexes are 0–2 and valid column indexes are 0–1.

Example 1: 2 x 2 matrix

Creates four integer elements.

```
int matrix[2][2]={{1,2},{3,4}};
```

Example 2: Class marks

Five students and three subjects.

```
int marks[5][3];
```

Example 3: Monthly sales

Four products across twelve months.

```
double sales[4][12];
```

Example 4: Seating plan

Six rows and ten seats.

```
bool seats[6][10];
```

Example 5: Access one cell

Select row 2, column 1.

```
cout << marks[2][1];
```

Example 6: Modify one cell

Changes one table entry.

```
marks[1][2]=95;
```

Example 7: Print table

Nested loops traverse rows and columns.

```
for(int r=0;r<5;++r){ for(int c=0;c<3;++c) cout<<marks[r][c]<<' '; cout<<'\n'; }
```

Example 8: Sum all cells

Accumulates every cell.

```
int sum=0; for(int r=0;r<5;++r) for(int c=0;c<3;++c) sum+=marks[r][c];
```

Example 9: Row total

Calculates one student's total.

```
int total=0; for(int c=0;c<3;++c) total+=marks[2][c];
```

Example 10: Column total

Calculates one subject's total.

```
int total=0; for(int r=0;r<5;++r) total+=marks[r][1];
```

5.2 Matrix operations

Example 1: Matrix addition

Adds corresponding cells of equal-size matrices.

```
for(int r=0;r<2;++r) for(int c=0;c<2;++c) C[r][c]=A[r][c]+B[r][c];
```

Example 2: Matrix subtraction

Subtracts corresponding cells.

```
for(int r=0;r<2;++r) for(int c=0;c<2;++c) C[r][c]=A[r][c]-B[r][c];
```

Example 3: Scalar multiplication

Multiplies each cell by a scalar.

```
for(int r=0;r<2;++r) for(int c=0;c<3;++c) A[r][c]*=5;
```

Example 4: Main diagonal

Reads A[i][i] in a square matrix.

```
for(int i=0;i<3;++i) cout<<A[i][i]<<' ';
```

Example 5: Transpose

Writes rows as columns.

```
for(int r=0;r<2;++r) for(int c=0;c<3;++c) T[c][r]=A[r][c];
```

Example 6: Row maximum

Finds largest value in one row.

```
int m=A[0][0]; for(int c=1;c<3;++c) if(A[0][c]>m)m=A[0][c];
```

Example 7: Column minimum

Finds smallest value in a column.

```
int m=A[0][1]; for(int r=1;r<4;++r) if(A[r][1]<m)m=A[r][1];
```

Example 8: Count zeros

Counts cells equal to zero.

```
int z=0; for(int r=0;r<3;++r) for(int c=0;c<3;++c) if(A[r][c]==0)++z;
```

Example 9: Identity matrix test

Checks diagonal 1 and off-diagonal 0.

```
bool ok=true; for(int r=0;r<3;++r) for(int c=0;c<3;++c) if(A[r][c] != (r==c)) ok=false;
```

Example 10: Display formatted matrix

Prints a matrix with rows.

```
for(int r=0;r<3;++r){ for(int c=0;c<3;++c) cout<<A[r][c]<<"\t"; cout<<'\n'; }
```

Chapter 6 — Searching and Sorting

6.1 Linear search

Linear search checks elements sequentially. It works on unsorted arrays and has worst-case time complexity O(n).

Example 1: Find first occurrence

Stops after the first match.

```
int pos=-1; for(int i=0;i<n;++i) if(a[i]==target){pos=i;break;}
```

Example 2: Report found/not found

Tests the resulting position.

```
if(pos!=-1) cout<<"Found"; else cout<<"Not found";
```

Example 3: Count occurrences

Does not stop after the first match.

```
int count=0; for(int x:a) if(x==target) ++count;
```

Example 4: Find last occurrence

Continues and overwrites the position.

```
int pos=-1; for(int i=0;i<n;++i) if(a[i]==target) pos=i;
```

Example 5: Search a range

Checks whether any value lies in a range.

```
bool found=false; for(int x:a) if(x>=50&&x<=60) found=true;
```

Example 6: Search maximum

Finds the index of maximum.

```
int p=0; for(int i=1;i<n;++i) if(a[i]>a[p]) p=i;
```

Example 7: Search minimum

Finds the index of minimum.

```
int p=0; for(int i=1;i<n;++i) if(a[i]<a[p]) p=i;
```

Example 8: Find duplicate

Detects a repeated value with nested loops.

```
bool dup=false; for(int i=0;i<n;++i) for(int j=i+1;j<n;++j) if(a[i]==a[j]) dup=true;
```

Example 9: Search with early termination

Breaks immediately when target appears.

```
for(int i=0;i<n;++i){ if(a[i]==target){cout<<i;break;} }
```

Example 10: Search multiple targets

Searches a second array of requested values.

```
for(int t:targets){ bool found=false; for(int x:a) if(x==t){found=true;break;} cout<<found; }
```

6.2 Sorting

Sorting rearranges elements according to an ordering rule. Bubble sort is useful for learning algorithmic mechanics, while std::sort is normally preferred in practical C++ programs.

Example 1: Bubble sort ascending

Repeatedly swaps adjacent elements that are out of order.

```
for(int i=0;i<n-1;++i) for(int j=0;j<n-1-i;++j) if(a[j]>a[j+1]) swap(a[j],a[j+1]);
```

Example 2: Bubble sort descending

Reverses the comparison.

```
for(int i=0;i<n-1;++i) for(int j=0;j<n-1-i;++j) if(a[j]<a[j+1]) swap(a[j],a[j+1]);
```

Example 3: std::sort ascending

Uses the standard library algorithm.

```
#include <algorithm>
sort(a,a+n);
```

Example 4: std::sort descending

Uses greater<int> as the comparator.

```
sort(a,a+n,greater<int>());
```

Example 5: Sort part of an array

Sorts a half-open range.

```
sort(a+2,a+n);
```

Example 6: Sort doubles

Works with floating-point values.

```
sort(price,price+n);
```

Example 7: Sort characters

Alphabetically orders character codes.

```
sort(letters,letters+n);
```

Example 8: Check sorted order

Verifies nondecreasing order.

```
bool sorted=true; for(int i=1;i<n;++i) if(a[i]<a[i-1]) sorted=false;
```

Example 9: Remove adjacent duplicates after sort

Sort first, then inspect neighboring values.

```
sort(a,a+n); for(int i=1;i<n;++i) if(a[i]==a[i-1]) cout<<a[i];
```

Example 10: Binary search after sorting

std::binary_search requires a sorted range.

```
sort(a,a+n); bool found=binary_search(a,a+n,target);
```

Chapter 7 — Arrays with Functions and Pointers

7.1 Passing arrays to functions

When a built-in array is passed to a function, the parameter is adjusted to a pointer to its first element. Therefore, the function does not automatically know the array's length. Pass the size explicitly or use a safer abstraction such as std::array or std::span in modern C++.

Example 1: Display array

Pass a pointer-like parameter and size.

```
void print(const int a[], size_t n){ for(size_t i=0;i<n;++i) cout<<a[i]<<' '; }
```

Example 2: Calculate sum

Returns the sum without modifying data.

```
int sum(const int a[], size_t n){ int s=0; for(size_t i=0;i<n;++i)s+=a[i]; return s; }
```

Example 3: Find maximum

Processes a read-only array.

```
int maxValue(const int a[],size_t n){int m=a[0];for(size_t i=1;i<n;++i)if(a[i]>m)m=a[i];return m;}
```

Example 4: Modify elements

Non-const parameter allows modification.

```
void doubleValues(int a[],size_t n){for(size_t i=0;i<n;++i)a[i]*=2;}
```

Example 5: Search

Returns the index of a target or -1.

```
int find(const int a[],size_t n,int t){for(size_t i=0;i<n;++i)if(a[i]==t)return i;return -1;}
```

Example 6: Average

Returns a double result.

```
double average(const int a[],size_t n){int s=0;for(size_t i=0;i<n;++i)s+=a[i];return(double)s/n;}
```

Example 7: Reverse

Swaps symmetric elements.

```
void reverseArray(int a[],size_t n){for(size_t i=0;i<n/2;++i)swap(a[i],a[n-1-i]);}
```

Example 8: Copy

Copies source into destination.

```
void copyArray(const int src[],int dst[],size_t n){for(size_t i=0;i<n;++i)dst[i]=src[i];}
```

Example 9: Function call

Passes an array and its element count.

```
int a[]={4,7,2}; cout<<sum(a,3);
```

Example 10: Const correctness

const prevents accidental modification.

```
void display(const double a[],size_t n){for(size_t i=0;i<n;++i)cout<<a[i]<<' ';}
```

7.2 Array and pointer relationship

The name of a built-in array often converts to a pointer to its first element in expressions. This is why pointer arithmetic can be used to traverse an array. However, an array itself is not a pointer; the distinction matters for sizeof, assignment, and type semantics.

Example 1: Pointer to first element

Obtains address of element zero.

```
int a[5]={1,2,3,4,5}; int* p=a;
```

Example 2: Dereference

Reads the first element through the pointer.

```
cout << *p;
```

Example 3: Pointer increment

Moves to the next element.

```
++p; cout << *p;
```

Example 4: Index through pointer

Equivalent pointer indexing is possible.

```
cout << p[2];
```

Example 5: Address of an element

Obtains the element address.

```
int* p2=&a[3];
```

Example 6: Modify through pointer

Changes the actual array element.

```
*p2=100;
```

Example 7: Traverse with pointer

Moves from first to last element.

```
for(int* p=a;p<a+5;++p) cout<<*p<<' ';
```

Example 8: Pointer difference

Distance between pointers gives element count between them.

```
ptrdiff_t d=&a[4]-&a[1];
```

Example 9: Array is not assignable

Two built-in arrays cannot be assigned directly.

```
int a[3], b[3];
// a = b; // invalid
```

Example 10: Pointer versus array size

sizeof produces different results.

```
int a[5]; int* p=a;
cout<<sizeof(a); // whole array
cout<<sizeof(p); // pointer
```

Chapter 8 — Character Arrays, Strings and Arrays of Structures

8.1 Character arrays

A character array can represent a sequence of characters. A C-style string is a character sequence terminated by the null character '\0'. For most application code, std::string is preferable because it manages length and storage more conveniently.

Example 1: Character sequence

Stores individual characters.

```
char code[4]={'A','B','C','D'};
```

Example 2: C-style string literal

The compiler includes a terminating null character.

```
char name[]="Mwamba";
```

Example 3: Access a character

Reads one character by index.

```
cout << name[0];
```

Example 4: Modify a character

Changes one character in a mutable array.

```
name[0]='m';
```

Example 5: Character count

Counts the characters before null terminator.

```
size_t n=strlen(name);
```

Example 6: Copy string

Uses the C library function.

```
char dest[20]; strcpy(dest,name);
```

Example 7: Compare strings

Uses strcmp for C-style strings.

```
if(strcmp(a,b)==0) cout<<"Equal";
```

Example 8: Find character

Uses strchr to locate a character.

```
char* p=strchr(name,'w');
```

Example 9: Use std::string

Modern C++ text representation.

```
#include <string>
std::string name="Mwamba";
```

Example 10: Array of strings

Stores several names using std::string.

```
std::string names[3]={"Alice","Brian","Chanda"};
```

8.2 Arrays of structures

A structure groups related fields into one object. An array of structures therefore represents a collection of records, which is common in student, employee, inventory and transaction systems.

Example 1: Student records

Creates three Student objects.

```
struct Student{int id; double mark;};
Student students[3];
```

Example 2: Initialize records

Uses aggregate initialization.

```
Student students[2]={{101,78.5},{102,84.0}};
```

Example 3: Access a field

Selects an object and then a field.

```
cout << students[0].mark;
```

Example 4: Update a field

Changes a particular record field.

```
students[1].mark=91.0;
```

Example 5: Traverse records

Processes every student.

```
for(const Student& s:students) cout<<s.id<<' '<<s.mark<<'\n';
```

Example 6: Find student by ID

Searches records sequentially.

```
for(const Student& s:students) if(s.id==102) cout<<s.mark;
```

Example 7: Calculate average

Aggregates a structure field.

```
double total=0; for(const Student& s:students) total+=s.mark;
```

Example 8: Array of inventory records

Stores product information.

```
struct Product{int id; double price; int stock;};
Product p[5];
```

Example 9: Update stock

Modifies one record.

```
p[2].stock += 10;
```

Example 10: Pass records to function

Passes an array and size.

```
void print(const Student a[],size_t n){for(size_t i=0;i<n;++i)cout<<a[i].id;}
```

Chapter 9 — Modern C++ Alternatives

9.1 std::array

std::array is a fixed-size container provided by the C++ standard library. Unlike a built-in array, it provides useful member functions such as size(), begin(), end(), and at(). It remains stack-friendly and has value semantics.

Example 1: Declare std::array

Requires the <array> header.

```
#include <array>
std::array<int,5> marks;
```

Example 2: Initialize

Uses aggregate initialization.

```
std::array<int,5> marks={70,80,65,90,75};
```

Example 3: Access with []

Fast unchecked indexing.

```
cout << marks[0];
```

Example 4: Bounds-checked access

at() throws std::out_of_range for invalid access.

```
cout << marks.at(0);
```

Example 5: Get size

size() reports element count.

```
cout << marks.size();
```

Example 6: Range loop

Works naturally with range-based for.

```
for(int x:marks) cout<<x<<' ';
```

Example 7: Sort

Works directly with standard algorithms.

```
sort(marks.begin(),marks.end());
```

Example 8: Fill

Assigns one value to every element.

```
marks.fill(0);
```

Example 9: Swap

Exchanges contents of two arrays.

```
a.swap(b);
```

Example 10: Pass to function

Pass by const reference without separate size.

```
void print(const std::array<int,5>& a){for(int x:a)cout<<x;}
```

9.2 std::vector

std::vector is the standard dynamic sequence container. It stores elements contiguously but can grow or shrink at runtime. For most modern programs where the number of elements is not known at compile time, vector is a better choice than a built-in variable-length array.

Example 1: Declare vector

Creates an initially empty vector.

```
#include <vector>
std::vector<int> marks;
```

Example 2: Initialize vector

Creates elements from an initializer list.

```
std::vector<int> marks={70,80,90};
```

Example 3: Append

Adds an element at the end.

```
marks.push_back(85);
```

Example 4: Remove last

Removes the final element.

```
marks.pop_back();
```

Example 5: Size

Returns current element count.

```
cout << marks.size();
```

Example 6: Bounds-checked access

at() checks the index.

```
cout << marks.at(1);
```

Example 7: Sort

Works directly with iterators.

```
sort(marks.begin(),marks.end());
```

Example 8: Iterate

Range-based for is convenient.

```
for(int x:marks) cout<<x<<' ';
```

Example 9: Clear

Removes all elements.

```
marks.clear();
```

Example 10: Reserve capacity

Can reduce reallocations when growth is predictable.

```
marks.reserve(100);
```

Chapter 10 — Errors, Debugging and Professional Practice

10.1 Common mistakes

Example 1: Off-by-one

Use < size, not <= size.

```
for(int i=0;i<5;++i) cout<<a[i];
```

Example 2: Invalid index

Index 5 is invalid for a five-element array.

```
int a[5];
// a[5]=10;
```

Example 3: Negative index

Negative indexes are invalid.

```
int a[5];
// cout<<a[-1];
```

Example 4: Uninitialized array

Local built-in elements are not automatically initialized to meaningful values.

```
int a[5];
// cout<<a[0];
```

Example 5: Wrong dimension

A 2D array needs both row and column indexes.

```
int a[3][2];
cout<<a[2][1];
```

Example 6: Integer division

Cast before division when a decimal average is required.

```
double avg=(double)sum/n;
```

Example 7: Size lost in function

Pass n when using built-in arrays.

```
void process(const int a[],size_t n);
```

Example 8: Buffer overflow

Ensure a character array has enough room, including '\0'.

```
char name[20];
// avoid copying more than 19 characters plus terminator
```

Example 9: Unnecessary magic numbers

Use named constants.

```
constexpr int STUDENTS=30;
int marks[STUDENTS];
```

Example 10: Missing headers

Include libraries for facilities you use.

```
#include <iostream>
#include <algorithm>
```

```
#include <array>
#include <vector>
```

10.2 Professional programming checklist

• Give arrays descriptive names such as marks, monthlySales, employeeIds and temperatures.

• Prefer named constants or constexpr values instead of unexplained numeric literals.

• Use const for arrays that a function should only read.

• Validate external input before storing or using it.

• Never access an index unless you have established that it is valid.

• Use std::array for fixed-size collections when modern C++ library facilities are useful.

• Use std::vector when the collection size changes at runtime.

• Break repeated array operations into functions with clear responsibilities.

• Use standard algorithms such as std::sort where they improve clarity and reliability.

• Test boundary cases: empty input, one element, minimum value, maximum value, duplicates and already-sorted data.

Chapter 11 — Integrated Practical Programs

Program 1: Student Marks Analysis

```
#include <iostream>
using namespace std;

int main() {
    const int N = 5;
    int marks[N];
    int total = 0;

    for (int i = 0; i < N; ++i) {
        cout << "Enter mark " << i + 1 << ": ";
        cin >> marks[i];
        total += marks[i];
    }

    int highest = marks[0];
    int lowest = marks[0];
    int pass = 0;

    for (int i = 1; i < N; ++i) {
        if (marks[i] > highest) highest = marks[i];
        if (marks[i] < lowest) lowest = marks[i];
    }

    for (int mark : marks)
        if (mark >= 50) ++pass;

    double average = static_cast<double>(total) / N;

    cout << "\nTotal: " << total << endl;
    cout << "Average: " << average << endl;
    cout << "Highest: " << highest << endl;
    cout << "Lowest: " << lowest << endl;
    cout << "Passed: " << pass << endl;
    cout << "Failed: " << N - pass << endl;

    return 0;
}
```

Program 2: Two-Dimensional Class Marks

```
#include <iostream>
using namespace std;

int main() {
    const int STUDENTS = 4;
    const int SUBJECTS = 3;
    int marks[STUDENTS][SUBJECTS];

    for (int r = 0; r < STUDENTS; ++r) {
        for (int c = 0; c < SUBJECTS; ++c) {
            cout << "Student " << r + 1
                 << ", subject " << c + 1 << ": ";
            cin >> marks[r][c];
        }
    }

    for (int r = 0; r < STUDENTS; ++r) {
        int total = 0;
        for (int c = 0; c < SUBJECTS; ++c)
            total += marks[r][c];

        cout << "Student " << r + 1
             << " total = " << total << endl;
    }

    return 0;
}
```

Program 3: Search and sort

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    int a[] = {45, 12, 89, 34, 67, 12};
    const int N = sizeof(a) / sizeof(a[0]);

    sort(a, a + N);

    cout << "Sorted: ";
    for (int x : a) cout << x << ' ';

    int target;
    cout << "\nEnter value to search: ";
    cin >> target;

    bool found = binary_search(a, a + N, target);

    cout << (found ? "Value found." : "Value not found.") << endl;
    return 0;
}
```

Chapter 12 — 25 Practical Scenario Questions

These tasks are designed as programming exercises rather than short-answer theory questions. For each task, produce a complete C++ program, use meaningful variable names, validate important input, and display clear output.

1. Student Performance Register

Store marks for 10 students. Display every mark, calculate the class total and average, identify the highest and lowest marks, and count passes and failures.

2. Monthly Sales Analysis

Store sales for 12 months. Calculate annual sales, average monthly sales, highest month, lowest month, and the month indexes where these occurred.

3. Employee Salary Analysis

Store salaries for 8 employees. Calculate total payroll, average salary, highest salary and number of employees earning above the average.

4. Inventory Management

Store quantities of 10 products. Display products with quantities below 5, calculate total inventory, and identify the product with the largest quantity.

5. Temperature Monitoring

Store seven daily temperature readings. Find the maximum, minimum, average and number of days above the weekly average.

6. Number Classification

Read 15 integers. Count positive, negative and zero values, then separately count even and odd values.

7. Linear Search System

Store 12 student identification numbers. Ask for an ID and report whether it exists and its position. If duplicate IDs exist, report all matching positions.

8. Frequency Analysis

Read 20 integers and determine the frequency of a value entered by the user. Explain the algorithm's time cost in comments.

9. Array Reversal

Read 10 integers and reverse the array in place using swaps. Display the array before and after reversal.

10. Array Copy and Comparison

Read two arrays of five integers. Copy the first into a third array and determine whether the original two arrays contain exactly the same values.

11. Ascending Sort

Read 10 integers and sort them using bubble sort. Display the number of swaps performed.

12. Descending Sort with std::sort

Read 10 values and use std::sort with an appropriate comparator to produce descending order.

13. Top Three Results

Store 15 marks and determine the highest three distinct values. Handle duplicate marks correctly.

14. Second Highest

Store 10 integers and determine the second-largest distinct value. Explain what happens if fewer than two distinct values exist.

15. Average Excluding Extremes

Store 10 marks and calculate an average after excluding the highest and lowest value.

16. Two-Dimensional Marks

Create a 2D array for 6 students and 4 subjects. Input all marks and calculate each student's total and average.

17. Subject Performance

Using the same 2D array, calculate the total and average for each subject and identify the best-performing subject.

18. Attendance Matrix

Create a 2D Boolean array representing 5 students over 7 days. Count attendance days for each student and identify the student with the highest attendance.

19. Sales Matrix

Create a 2D array representing 4 products over 6 months. Calculate total sales per product and total sales per month.

20. Matrix Addition

Read two 3×3 integer matrices and produce their sum. Display all three matrices in a professional tabular format.

21. Matrix Transpose

Read a 3×4 matrix and create its transpose as a 4×3 matrix. Display both matrices.

22. Array of Structures — Students

Define a Student structure containing ID, name and mark. Store 5 students in an array, then search for a student by ID and display the record.

23. Array of Structures — Products

Define a Product structure containing code, name, price and quantity. Store 8 products and calculate total inventory value for each product.

24. Modern C++ Comparison

Implement the same fixed-size marks problem once with a built-in array and once with std::array. Compare the syntax and available member functions.

25. Mini Academic Analytics System

Build a complete program using arrays that accepts 20 marks, validates them, displays a report, calculates mean/minimum/maximum, counts grade categories, sorts the marks, searches for a requested score, and prints a concise statistical summary.

Chapter 13 — Model Solution Patterns

The following patterns are not intended to replace your own solutions. They demonstrate professional approaches that can be adapted to the 25 scenarios.

Pattern A — Input and traversal

Use this structure whenever the task requires entering and displaying N values.

```
const int N = 10;
int a[N];

for (int i = 0; i < N; ++i) {
    cin >> a[i];
}

for (int i = 0; i < N; ++i) {
    cout << a[i] << ' ';
}
```

Pattern B — Aggregate statistics

Initialize extrema from the first element rather than arbitrary values.

```
int total = a[0];
int maximum = a[0];
int minimum = a[0];

for (int i = 1; i < N; ++i) {
    total += a[i];
    if (a[i] > maximum) maximum = a[i];
    if (a[i] < minimum) minimum = a[i];
}

double average = static_cast<double>(total) / N;
```

Pattern C — Linear search

Use a sentinel such as -1 for an unsuccessful index search.

```
int position = -1;

for (int i = 0; i < N; ++i) {
    if (a[i] == target) {
        position = i;
        break;
    }
}

if (position == -1)
    cout << "Not found";
else
    cout << "Found at index " << position;
```

Pattern D — Bubble sort

Use adjacent comparisons and progressively reduce the inner-loop range.

```
for (int pass = 0; pass < N - 1; ++pass) {
    for (int j = 0; j < N - 1 - pass; ++j) {
        if (a[j] > a[j + 1]) {
            swap(a[j], a[j + 1]);
        }
    }
}
```

Pattern E — 2D traversal

The outer loop controls rows and the inner loop controls columns.

```
for (int row = 0; row < ROWS; ++row) {
    for (int col = 0; col < COLS; ++col) {
        cout << table[row][col] << ' ';
    }
    cout << '\n';
}
```

Pattern F — Row total

Hold the row constant and iterate through columns.

```
for (int row = 0; row < ROWS; ++row) {
    int total = 0;
    for (int col = 0; col < COLS; ++col)
        total += marks[row][col];

    cout << total << '\n';
}
```

Pattern G — Column total

Hold the column constant and iterate through rows.

```
for (int col = 0; col < COLS; ++col) {
    int total = 0;
    for (int row = 0; row < ROWS; ++row)
        total += marks[row][col];

    cout << total << '\n';
}
```

Pattern H — Array function

For a built-in array, pass the element count explicitly.

```
double average(const int a[], size_t n) {
    int total = 0;

    for (size_t i = 0; i < n; ++i)
        total += a[i];

    return static_cast<double>(total) / n;
}
```

Pattern I — std::array

Use its size and iterators rather than managing the size manually.

```
std::array<int, 5> a = {4, 1, 7, 2, 9};

sort(a.begin(), a.end());

for (int x : a)
    cout << x << ' ';
```

Pattern J — std::vector

Use vector when the number of elements can change at runtime.

```
std::vector<int> values;

int n;
cin >> n;

for (int i = 0; i < n; ++i) {
    int x;
    cin >> x;
    values.push_back(x);
}
```

Chapter 14 — Revision Summary

A built-in array is a fixed-size, homogeneous sequence accessed by zero-based indexes. Its core syntax is data_type name[size]. One-dimensional arrays use one index; two-dimensional arrays use row and column indexes. Loops provide systematic traversal, while common processing patterns include aggregation, searching and sorting.

At a professional level, the programmer must also understand the limitations of built-in arrays: they do not know their own size when passed to ordinary functions, they provide no automatic bounds checking, and their fixed size can be inconvenient. Modern C++ therefore provides std::array for fixed-size collections and std::vector for dynamic collections.

For assessment work, demonstrate not only that the program produces the correct output, but also that the code is readable, the array boundaries are respected, input is handled responsibly, and the algorithm is appropriate for the task.

Key Syntax Reference

Purpose	Syntax
Declaration	`data_type name[size];`
Initialization	`data_type name[size] = {v1, v2, ...};`
Inferred size	`data_type name[] = {v1, v2, ...};`
Access	`name[index]`
Update	`name[index] = value;`
1D traversal	`for (int i=0; i<size; ++i)`
2D access	`name[row][column]`
2D traversal	`for (int r=0; r<rows; ++r) for (int c=0; c<cols; ++c)`
Element count	`sizeof(name) / sizeof(name[0])`
std::array	`std::array<T, N> name;`
std::vector	`std::vector<T> name;`
Sort built-in array	`std::sort(name, name + size);`
Sort std::array	`std::sort(name.begin(), name.end());`
Sort vector	`std::sort(name.begin(), name.end());`

End of study notes.